1. Apply data cleaning techniques on any dataset (e.g., Paper Reviews dataset in UCI repository). Techniques may include handling missing values, outliers and inconsistent values. A set of validation rules can be prepared based on the dataset and validations can be performed.

In [34]:
```
import pandas as pd
import json

with open('reviews.json', 'r', encoding='utf-8') as f:
    data = json.load(f)

df = pd.json_normalize(
    data['paper'],
    record_path=['review'],
    meta=['id', 'preliminary_decision'],
    record_prefix='review_',
    meta_prefix='paper_'
)

print(df.head())
```

```
  review_confidence review_evaluation  review_id review_lan  \
0                 4                 1          1         es
1                 4                 1          2         es
2                 5                 1          3         es
3                 4                 2          1         es
4                 4                 2          2         es

  review_orientation review_remarks  \
0                  0
1                  1
2                  1
3                  1
4                  0

                                        review_text review_timespan paper_id  \
0  - El artículo aborda un problema contingente y...      2010-07-05        1
1  El artículo presenta recomendaciones prácticas...      2010-07-05        1
2  - El tema es muy interesante y puede ser de mu...      2010-07-05        1
3  Se explica en forma ordenada y didáctica una e...      2010-07-05        2
4                                                     2010-07-05        2

  paper_preliminary_decision
0                     accept
1                     accept
2                     accept
3                     accept
4                     accept
```

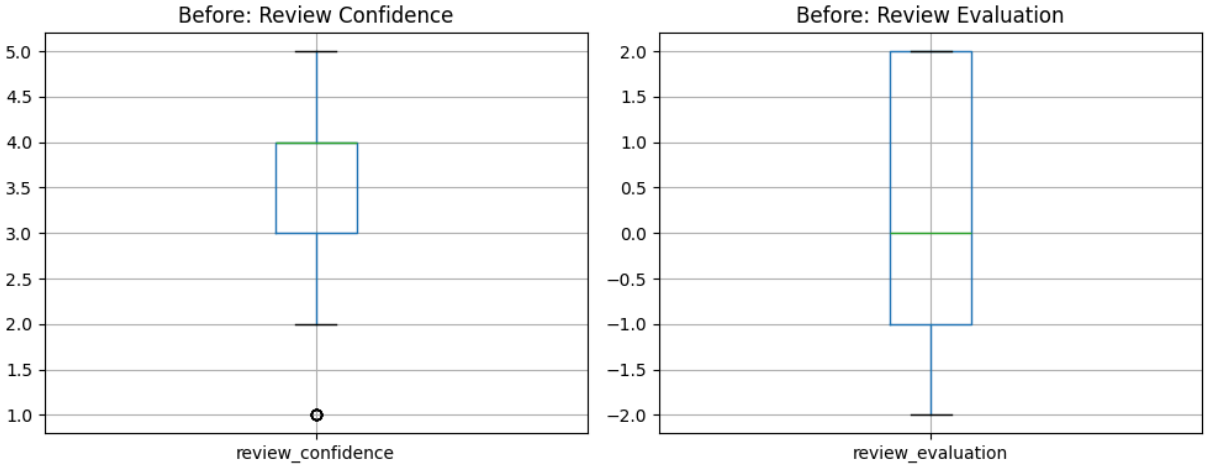
In [35]:
```
# ========= handling missing values =========

print(df.isnull().sum())
```

```
df['review_confidence'] = pd.to_numeric(df['review_confidence'], errors='coerce')
df['review_evaluation'] = pd.to_numeric(df['review_evaluation'], errors='coerce')

df.fillna({'review_confidence': df['review_confidence'].mean(), 'review_evaluation'

# df['review_remarks'].fillna('No remarks', inplace=True)
# df['review_text'].fillna('No text provided', inplace=True)

df.fillna({'review_remarks': '', 'review_text': ''}, inplace=True)
```

```
review_confidence             2
review_evaluation             0
review_id                     0
review_lan                    0
review_orientation            0
review_remarks                0
review_text                   0
review_timespan               0
paper_id                      0
paper_preliminary_decision    0
dtype: int64
```

In [36]:
```
# ========= handling inconsistent data =========

df['review_lan'] = df['review_lan'].str.strip().str.lower()
# df['review_lan'].replace({'spa': 'es', 'en-us': 'en'}, inplace=True)
df.replace({'review_lan': {'spa': 'es', 'en-us': 'en'}}, inplace=True)

df['paper_preliminary_decision'] = df['paper_preliminary_decision'].str.strip().str
df.loc[~df['paper_preliminary_decision'].isin(['accept', 'reject']), 'paper_prelimi
```

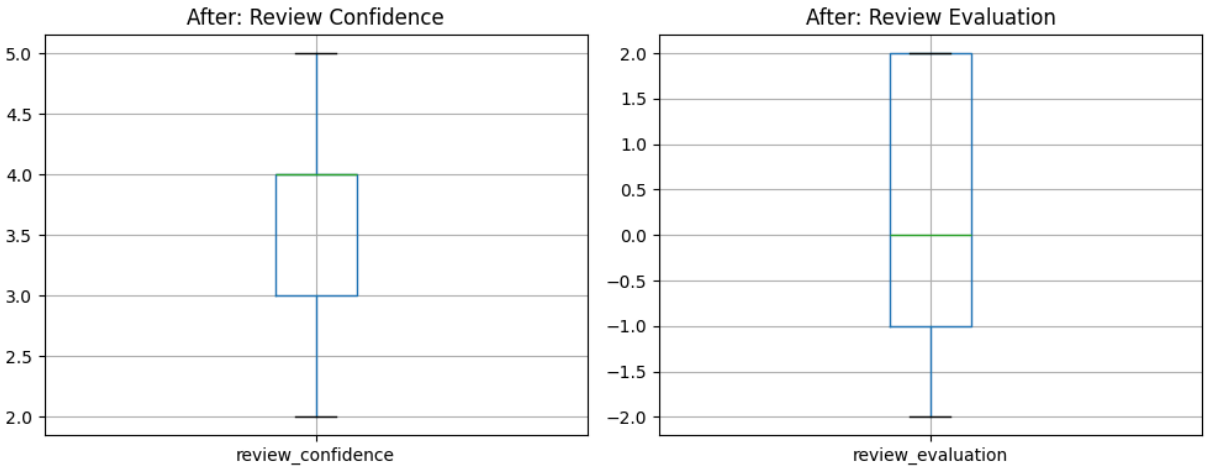
In [37]:
```
# Visualize outliers before handling
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
df.boxplot(column=['review_confidence'])
plt.title('Before: Review Confidence')

plt.subplot(1, 2, 2)
df.boxplot(column=['review_evaluation'])
plt.title('Before: Review Evaluation')
plt.tight_layout()
plt.show()
```



In [38]:
```python
# ========= handling outliers using IQR method =========

for col in ['review_confidence', 'review_evaluation']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"Outliers found in {col}: {len(outliers)}")

    # Remove outliers
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]

print(f"\nShape of DataFrame after removing outliers: {df.shape}")
```

```
Outliers found in review_confidence: 7
Outliers found in review_evaluation: 0

Shape of DataFrame after removing outliers: (398, 10)
```

In [39]:
```python
# Visualize data after handling outliers
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
df.boxplot(column=['review_confidence'])
plt.title('After: Review Confidence')

plt.subplot(1, 2, 2)
df.boxplot(column=['review_evaluation'])
plt.title('After: Review Evaluation')
plt.tight_layout()
plt.show()
```



Validation Rules:

Rule	Field	Expected Range / Type
R1	review_confidence	1–5
R2	review_evaluation	-2–2
R3	review_lan	one of ['en', 'es', 'fr']
R4	paper_preliminary_decision	['accept', 'reject']
R5	review_text	not empty

In [42]:
```
# ========= applying validation rules =========

rule1 = df["review_confidence"].between(1, 5)
rule2 = df["review_evaluation"].between(-2, 2)
rule3 = df["review_lan"].isin(["en", "es", "fr"])
rule4 = df["paper_preliminary_decision"].isin(["accept", "reject"])
rule5 = df["review_text"].str.strip().ne("")

Rules = pd.DataFrame({
    "Rule 1 (Confidence 1–5)": rule1,
    "Rule 2 (Evaluation -2–2)": rule2,
    "Rule 3 (Language Valid)": rule3,
    "Rule 4 (Decision Valid)": rule4,
    "Rule 5 (Text Non-empty)": rule5
})

print("Rule Violation Summary:")
for col in Rules.columns:
    total = len(Rules)
    passed = Rules[col].sum()
    failed = total - passed
    print(f"{col}: {failed} violations out of {total}")
```

```
Rule Violation Summary:
Rule 1 (Confidence 1–5): 0 violations out of 398
Rule 2 (Evaluation -2–2): 0 violations out of 398
Rule 3 (Language Valid): 0 violations out of 398
Rule 4 (Decision Valid): 21 violations out of 398
Rule 5 (Text Non-empty): 6 violations out of 398
```

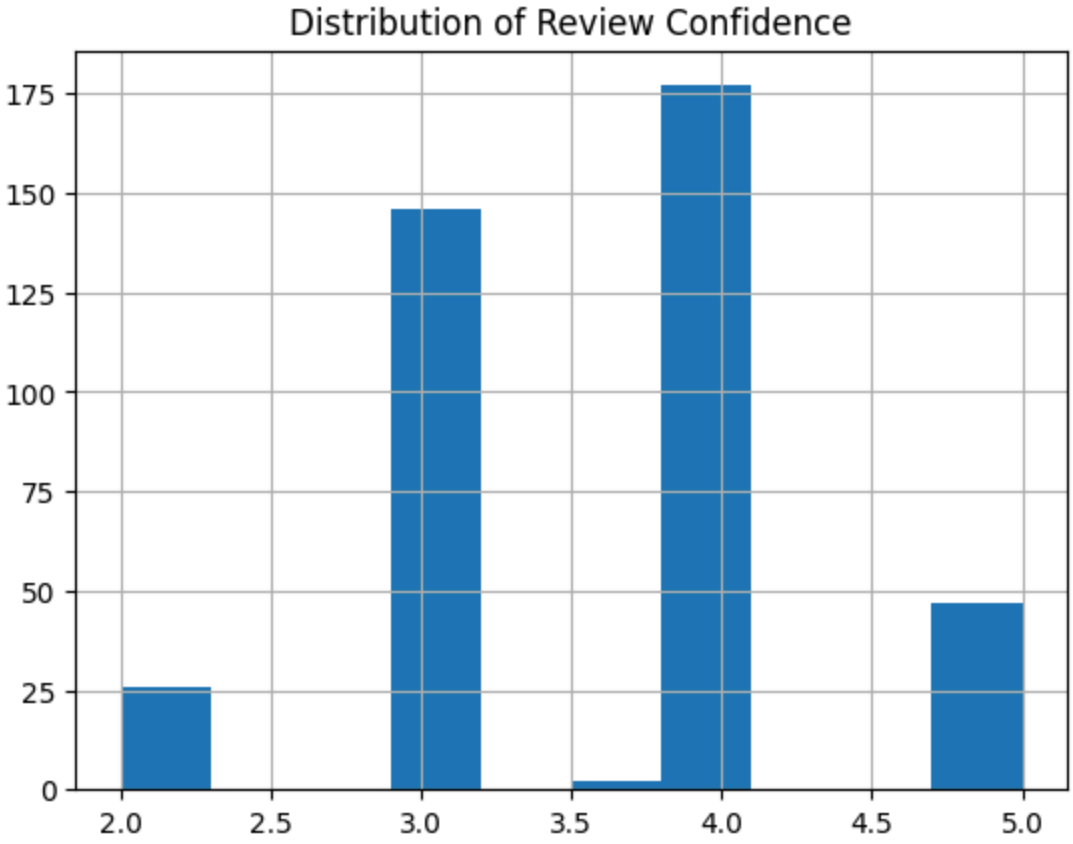
In [41]:

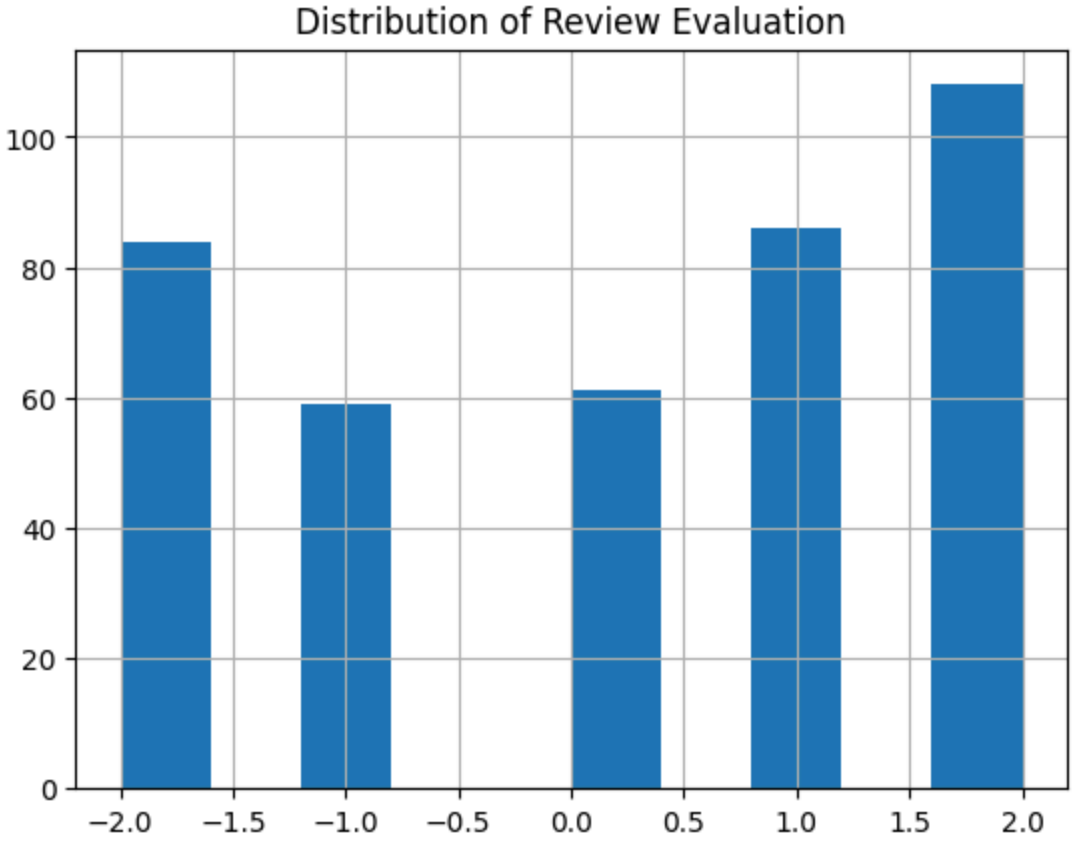
```python
# ========= final cleaned data =========

print(df.describe(include='all'))

# Visualize numeric distributions
import matplotlib.pyplot as plt
df['review_confidence'].hist()
plt.title('Distribution of Review Confidence')
plt.show()

df['review_evaluation'].hist()
plt.title('Distribution of Review Evaluation')
plt.show()

# save cleaned data
df.to_csv("cleaned_paper_reviews.csv", index=False)
```

	review_confidence	review_evaluation	review_id	review_lan	\
count	398.000000	398.000000	398.000000	398	
unique	NaN	NaN	NaN	2	
top	NaN	NaN	NaN	es	
freq	NaN	NaN	NaN	381	
mean	3.618458	0.188442	1.819095	NaN	
std	0.776587	1.504709	0.816972	NaN	
min	2.000000	-2.000000	1.000000	NaN	
25%	3.000000	-1.000000	1.000000	NaN	
50%	4.000000	0.000000	2.000000	NaN	
75%	4.000000	2.000000	2.000000	NaN	
max	5.000000	2.000000	4.000000	NaN	

	review_orientation	review_remarks	review_text	review_timespan	\
count	398	398	398	398	
unique	5	108	393	4	
top	-1			2014-07-05	
freq	139	288	6	124	
mean	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	
25%	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	
75%	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	

	paper_id	paper_preliminary_decision
count	398.0	377
unique	168.0	2
top	112.0	accept
freq	4.0	257
mean	NaN	NaN
std	NaN	NaN
min	NaN	NaN
25%	NaN	NaN
50%	NaN	NaN
75%	NaN	NaN
max	NaN	NaN


Distribution of Review Confidence
175
150
125
100
75
50
25
0
2.0
2.5
3.0
3.5
4.0
4.5
5.0


Distribution of Review Evaluation
100
80
60
40
20
0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

2. Apply data pre-processing techniques such as standardization/normalization, transformation, aggregation, discretization/binarization, sampling etc. on any dataset

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler, MinMaxScaler, Binarizer

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(df.head())
print("\nDataset shape:", df.shape)
print("\nSummary statistics:\n", df.describe())

X = df.drop(columns=['species'])
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa

Dataset shape: (150, 5)

Summary statistics:
        sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)
count        150.000000
mean           1.199333
std            0.762238
min            0.100000
25%            0.300000
50%            1.300000
75%            1.800000
max            2.500000
```

In [2]:
```
scaler = StandardScaler()
df_standardized = scaler.fit_transform(X)
df_standardized = pd.DataFrame(df_standardized, columns=X.columns)

print("\nStandardized Data (first 5 rows):\n", df_standardized.head())
```

```
Standardized Data (first 5 rows):
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0          -0.900681          1.019004          -1.340227         -1.315444
1          -1.143017         -0.131979          -1.340227         -1.315444
2          -1.385353          0.328414          -1.397064         -1.315444
3          -1.506521          0.098217          -1.283389         -1.315444
4          -1.021849          1.249201          -1.340227         -1.315444
```

In [3]:
```
normalizer = MinMaxScaler()
df_normalized = normalizer.fit_transform(X)
df_normalized = pd.DataFrame(df_normalized, columns=X.columns)

print("\nNormalized Data (first 5 rows):\n", df_normalized.head())
```

```
Normalized Data (first 5 rows):
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0           0.222222          0.625000           0.067797          0.041667
1           0.166667          0.416667           0.067797          0.041667
2           0.111111          0.500000           0.050847          0.041667
3           0.083333          0.458333           0.084746          0.041667
4           0.194444          0.666667           0.067797          0.041667
```

In [4]:
```
df_transformed = df.copy()
df_transformed["sepal width (cm)"] = np.log1p(df_transformed["sepal width (cm)"])

print("\nTransformed (log) Data (first 5 rows):\n", df_transformed.head())
```

```
Transformed (log) Data (first 5 rows):
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1          1.504077                1.4               0.2
1                4.9          1.386294                1.4               0.2
2                4.7          1.435085                1.3               0.2
3                4.6          1.410987                1.5               0.2
4                5.0          1.526056                1.4               0.2

  species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa
```

In [5]:
```
agg_df = df.groupby("species", observed=False).mean()
print("\nAggregated Mean by Species:\n", agg_df)
```

```
Aggregated Mean by Species:
            sepal length (cm)  sepal width (cm)  petal length (cm)  \
species
setosa                  5.006             3.428              1.462
versicolor              5.936             2.770              4.260
virginica               6.588             2.974              5.552

            petal width (cm)
species
setosa                 0.246
versicolor             1.326
virginica              2.026
```

In [6]:
```
bins = [0, 2.5, 5, 7.5]
labels = ['Small', 'Medium', 'Large']

df_discrete = df.copy()
df_discrete['custom_petal_length'] = pd.cut(df['petal length (cm)'], bins=bins, lab

print("\nDiscretized Data (Petal Length into 3 bins):\n", df_discrete.head())
```

```
Discretized Data (Petal Length into 3 bins):
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  species custom_petal_length
0  setosa               Small
1  setosa               Small
2  setosa               Small
3  setosa               Small
4  setosa               Small
```

In [7]:
```
binarizer = Binarizer(threshold=1.0)
df_binarized = df.copy()
df_binarized["wide_petal"] = binarizer.fit_transform(df[["petal width (cm)"]])

print("\nBinarized Data (Petal width >= 1.0 -> 1):\n", df_binarized.head())
```

```
Binarized Data (Petal width >= 1.0 -> 1):
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  species  wide_petal
0  setosa         0.0
1  setosa         0.0
2  setosa         0.0
3  setosa         0.0
4  setosa         0.0
```

In [8]:
```
sample_df = df.sample(frac=0.2, random_state=42)
print("\nSampled 20% of the Data:\n", sample_df)
```

Sampled 20% of the Data:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
73	6.1	2.8	4.7	1.2	
18	5.7	3.8	1.7	0.3	
118	7.7	2.6	6.9	2.3	
78	6.0	2.9	4.5	1.5	
76	6.8	2.8	4.8	1.4	
31	5.4	3.4	1.5	0.4	
64	5.6	2.9	3.6	1.3	
141	6.9	3.1	5.1	2.3	
68	6.2	2.2	4.5	1.5	
82	5.8	2.7	3.9	1.2	
110	6.5	3.2	5.1	2.0	
12	4.8	3.0	1.4	0.1	
36	5.5	3.5	1.3	0.2	
9	4.9	3.1	1.5	0.1	
19	5.1	3.8	1.5	0.3	
56	6.3	3.3	4.7	1.6	
104	6.5	3.0	5.8	2.2	
69	5.6	2.5	3.9	1.1	
55	5.7	2.8	4.5	1.3	
132	6.4	2.8	5.6	2.2	
29	4.7	3.2	1.6	0.2	
127	6.1	3.0	4.9	1.8	
26	5.0	3.4	1.6	0.4	
128	6.4	2.8	5.6	2.1	
131	7.9	3.8	6.4	2.0	
145	6.7	3.0	5.2	2.3	
108	6.7	2.5	5.8	1.8	
143	6.8	3.2	5.9	2.3	
45	4.8	3.0	1.4	0.3	
30	4.8	3.1	1.6	0.2	

	species
73	versicolor
18	setosa
118	virginica
78	versicolor
76	versicolor
31	setosa
64	versicolor
141	virginica
68	versicolor
82	versicolor
110	virginica
12	setosa
36	setosa
9	setosa
19	setosa
56	versicolor
104	virginica
69	versicolor
55	versicolor
132	virginica
29	setosa
127	virginica

```
26       setosa
128   virginica
131   virginica
145   virginica
108   virginica
143   virginica
45       setosa
30       setosa
```

3. Run Apriori algorithm to find frequent item sets and association rules on 2 real datasets and use appropriate evaluation measures to compute correctness of obtained patterns

- a) Use minimum support as 50% and minimum confidence as 75%
- b) Use minimum support as 60% and minimum confidence as 60 %

In [1]:
```
from efficient_apriori import apriori
import pandas as pd
import numpy as np

groceries = pd.read_csv("groceries.csv", header=0)
print(groceries.head())
print(groceries.shape)

bakery = pd.read_csv("bakery.csv")
bakery.columns = bakery.columns.str.strip()
print(bakery.head())
print(bakery.shape)
```

```
   Item(s)            Item 1               Item 2          Item 3  \
0        4       citrus fruit  semi-finished bread       margarine
1        3    tropical fruit               yogurt          coffee
2        1        whole milk                  NaN             NaN
3        4         pip fruit               yogurt    cream cheese
4        4  other vegetables            whole milk  condensed milk

                    Item 4 Item 5 Item 6 Item 7 Item 8 Item 9  ... Item 23  \
0              ready soups    NaN    NaN    NaN    NaN    NaN  ...     NaN
1                      NaN    NaN    NaN    NaN    NaN    NaN  ...     NaN
2                      NaN    NaN    NaN    NaN    NaN    NaN  ...     NaN
3             meat spreads    NaN    NaN    NaN    NaN    NaN  ...     NaN
4  long life bakery product    NaN    NaN    NaN    NaN    NaN  ...     NaN

  Item 24 Item 25 Item 26 Item 27 Item 28 Item 29 Item 30 Item 31 Item 32
0     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN
1     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN
2     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN
3     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN
4     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN     NaN

[5 rows x 33 columns]
(9835, 33)
   ID  Chocolate Cake  Lemon Cake  Casino Cake  Opera Cake  Strawberry Cake  \
0   1               0           0            0           0                1
1   2               0           1            0           0                0
2   3               0           0            0           0                0
3   4               0           0            0           0                0
4   5               0           0            0           0                0

   Truffle Cake  Chocolate Eclair  Coffee Eclair  Vanilla Eclair  ...  \
0             1                 1              0               0  ...
1             0                 0              0               0  ...
2             0                 0              0               0  ...
3             0                 0              0               0  ...
4             0                 0              0               0  ...

   Lemon Lemonade  Raspberry Lemonade  Orange Juice  Green Tea  Bottled Water  \
0               0                   0             0          0              0
1               0                   0             0          0              0
2               0                   0             0          0              0
3               0                   0             0          0              0
4               0                   0             0          0              0

   Hot Coffee  Chocolate Coffee  Vanilla Frappucino  Cherry Soda  \
0           0                 0                   0            0
1           0                 1                   1            0
2           0                 0                   0            0
3           0                 0                   0            0
4           0                 0                   0            0

   Single Espresso
0                0
1                0
2                0
3                0
```

```
4                0

[5 rows x 51 columns]
(5000, 51)
```

In [2]:
```python
transactions = []
for i in range(len(groceries)):
    row = groceries.iloc[i, 1:].dropna().astype(str).tolist()
    transactions.append(row)

print(len(transactions), transactions[:5])

transactions_b = []
item_columns = bakery.columns[1:]
for i in range(len(bakery)):
    items = item_columns[bakery.iloc[i, 1:] == 1].tolist()
    if items:
        transactions_b.append(items)

print(len(transactions_b), transactions_b[:5])
```

```
9835 [['citrus fruit', 'semi-finished bread', 'margarine', 'ready soups'], ['tropica
l fruit', 'yogurt', 'coffee'], ['whole milk'], ['pip fruit', 'yogurt', 'cream chees
e', 'meat spreads'], ['other vegetables', 'whole milk', 'condensed milk', 'long life
bakery product']]
5000 [['Strawberry Cake', 'Truffle Cake', 'Chocolate Eclair', 'Almond Tart'], ['Lemo
n Cake', 'Apple Tart', 'Lemon Tart', 'Chocolate Coffee', 'Vanilla Frappucino'], ['Bl
ueberry Tart', 'Apricot Croissant'], ['Cherry Tart', 'Lemon Cookie', 'Apricot Danis
h'], ['Apple Tart', 'Gongolais Cookie', 'Marzipan Cookie', 'Almond Croissant']]
```

In [3]:
```python
itemsets, rules = apriori(
    transactions,
    min_support=0.05,
    min_confidence=0.25
)

print("Frequent Itemsets (Groceries):")
print(itemsets)
print("\nRules (Groceries):")
print(rules)

b_item, b_rules = apriori(
    transactions_b,
    min_support=0.05,
    min_confidence=0.25
)

print("\nFrequent Itemsets (Bakery):")
print(b_item)
print("\nRules (Bakery):")
print(b_rules)
```

Frequent Itemsets (Groceries):
{1: {('citrus fruit',): 814, ('margarine',): 576, ('tropical fruit',): 1032, ('yogurt',): 1372, ('coffee',): 571, ('whole milk',): 2513, ('pip fruit',): 744, ('other vegetables',): 1903, ('butter',): 545, ('rolls/buns',): 1809, ('bottled beer',): 792, ('bottled water',): 1087, ('curd',): 524, ('beef',): 516, ('frankfurter',): 580, ('soda',): 1715, ('fruit/vegetable juice',): 711, ('newspapers',): 785, ('pastry',): 875, ('root vegetables',): 1072, ('canned beer',): 764, ('sausage',): 924, ('brown bread',): 638, ('shopping bags',): 969, ('napkins',): 515, ('pork',): 567, ('whipped/sour cream',): 705, ('domestic eggs',): 624}, 2: {('other vegetables', 'whole milk'): 736, ('rolls/buns', 'whole milk'): 557, ('whole milk', 'yogurt'): 551}}

Rules (Groceries):
[{whole milk} -> {other vegetables}, {other vegetables} -> {whole milk}, {rolls/buns} -> {whole milk}, {yogurt} -> {whole milk}]

Frequent Itemsets (Bakery):
{1: {('Strawberry Cake',): 480, ('Truffle Cake',): 438, ('Lemon Cake',): 425, ('Apple Tart',): 367, ('Lemon Tart',): 354, ('Chocolate Coffee',): 404, ('Vanilla Frappuccino',): 367, ('Blueberry Tart',): 426, ('Apricot Croissant',): 411, ('Cherry Tart',): 460, ('Lemon Cookie',): 321, ('Apricot Danish',): 446, ('Gongolais Cookie',): 477, ('Marzipan Cookie',): 418, ('Walnut Cookie',): 353, ('Chocolate Cake',): 399, ('Casino Cake',): 373, ('Berry Tart',): 391, ('Lemon Lemonade',): 324, ('Bottled Water',): 386, ('Opera Cake',): 418, ('Tuile Cookie',): 499, ('Apple Croissant',): 371, ('Apple Danish',): 391, ('Cherry Soda',): 335, ('Chocolate Tart',): 381, ('Coffee Eclair',): 554, ('Apple Pie',): 391, ('Almond Twist',): 407, ('Orange Juice',): 461, ('Raspberry Lemonade',): 339, ('Napolean Cake',): 409, ('Hot Coffee',): 513, ('Raspberry Cookie',): 320, ('Green Tea',): 310, ('Cheese Croissant',): 385, ('Blackberry Tart',): 380, ('Single Espresso',): 327}, 2: {('Apricot Danish', 'Cherry Tart'): 256}}

Rules (Bakery):
[{Cherry Tart} -> {Apricot Danish}, {Apricot Danish} -> {Cherry Tart}]

4. Use Naive bayes, K-nearest, and Decision tree classification algorithms to build classifiers on any two datasets. Pre-process the datasets using techniques specified in Q2. Compare the Accuracy, Precision, Recall and F1 measure reported for each dataset using the abovementioned classifiers under the following situations: i. Using Holdout method (Random sampling):

- a) Training set = 80% Test set = 20%
- b) Training set = 66.6% (2/3rd of total), Test set = 33.3% ii. Using Cross-Validation:
  - a) 10-fold
  - b) 5-fold

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split, cross_validate, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

In [2]:
```
iris = load_iris()
X1 = pd.DataFrame(iris.data, columns=iris.feature_names)
y1 = iris.target

wine = load_wine()
X2 = pd.DataFrame(wine.data, columns=wine.feature_names)
y2 = wine.target
```

In [3]:
```
scaler = StandardScaler()
X1_scaled = scaler.fit_transform(X1)
X2_scaled = scaler.fit_transform(X2)

models = {
    "Naive Bayes": GaussianNB(),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}

def evaluate_model(model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    return {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, average='weighted'),
        "Recall": recall_score(y_test, y_pred, average='weighted'),
```

```
        "F1-score": f1_score(y_test, y_pred, average='weighted')
    }
```

In [4]:
```
# Holdout Validation

holdout_results = []

for dataset_name, (X, y) in {"Iris": (X1_scaled, y1), "Wine": (X2_scaled, y2)}.item
    for split, test_size in {"80/20": 0.2, "66.6/33.3": 0.333}.items():
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_si

        for model_name, model in models.items():
            scores = evaluate_model(model, X_train, X_test, y_train, y_test)
            holdout_results.append({
                "Dataset": dataset_name,
                "Split": split,
                "Model": model_name,
                **scores
            })

holdout_df = pd.DataFrame(holdout_results)
print("\n=== Holdout Method Results ===")
print(holdout_df)
```

```
=== Holdout Method Results ===
   Dataset      Split          Model  Accuracy  Precision    Recall  F1-score
0     Iris      80/20    Naive Bayes  0.966667   0.969697  0.966667  0.966583
1     Iris      80/20            KNN  0.933333   0.944444  0.933333  0.932660
2     Iris      80/20  Decision Tree  0.933333   0.933333  0.933333  0.933333
3     Iris  66.6/33.3    Naive Bayes  0.920000   0.923649  0.920000  0.919722
4     Iris  66.6/33.3            KNN  0.920000   0.935238  0.920000  0.918877
5     Iris  66.6/33.3  Decision Tree  0.940000   0.949000  0.940000  0.939529
6     Wine      80/20    Naive Bayes  0.972222   0.974359  0.972222  0.972263
7     Wine      80/20            KNN  0.972222   0.974747  0.972222  0.972369
8     Wine      80/20  Decision Tree  0.944444   0.951389  0.944444  0.944961
9     Wine  66.6/33.3    Naive Bayes  0.966667   0.969231  0.966667  0.966456
10    Wine  66.6/33.3            KNN  0.950000   0.957895  0.950000  0.950476
11    Wine  66.6/33.3  Decision Tree  0.983333   0.984000  0.983333  0.983290
```

In [5]:
```
# Cross-Validation

cv_results = []

for dataset_name, (X, y) in {"Iris": (X1_scaled, y1), "Wine": (X2_scaled, y2)}.item
    for folds in [5, 10]:
        kf = KFold(n_splits=folds, shuffle=True, random_state=42)

        for model_name, model in models.items():
            scores = cross_validate(model, X, y, cv=kf,
                                    scoring=['accuracy', 'precision_weighted', 'rec

            cv_results.append({
                "Dataset": dataset_name,
                "CV_Folds": folds,
                "Model": model_name,
                "Accuracy": np.mean(scores['test_accuracy']),
```

```
                "Precision": np.mean(scores['test_precision_weighted']),
                "Recall": np.mean(scores['test_recall_weighted']),
                "F1-score": np.mean(scores['test_f1_weighted'])
            })

cv_df = pd.DataFrame(cv_results)
print("\n=== Cross-Validation Results ===")
print(cv_df)
```

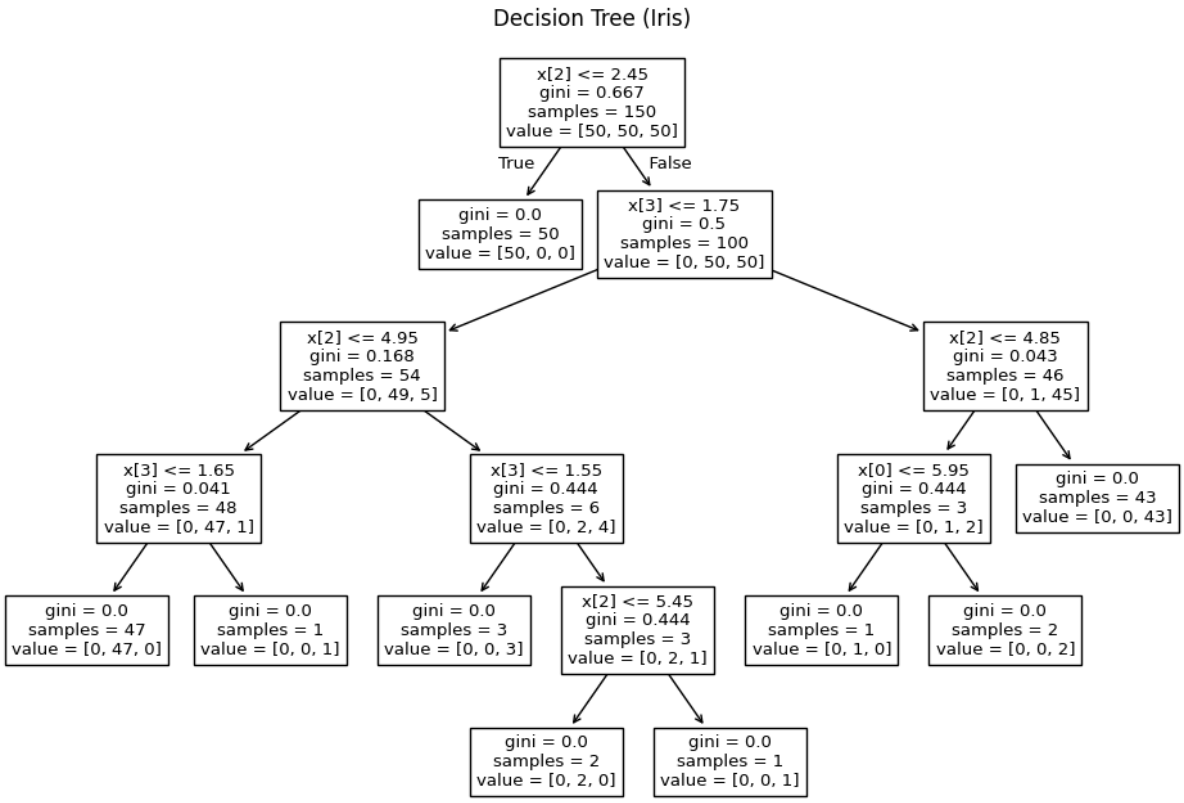
```
=== Cross-Validation Results ===
   Dataset  CV_Folds          Model  Accuracy  Precision    Recall  F1-score
0     Iris         5    Naive Bayes  0.960000   0.963879  0.960000  0.959916
1     Iris         5            KNN  0.960000   0.962187  0.960000  0.959797
2     Iris         5  Decision Tree  0.953333   0.958657  0.953333  0.953185
3     Iris        10    Naive Bayes  0.960000   0.967619  0.960000  0.960287
4     Iris        10            KNN  0.953333   0.960254  0.953333  0.952893
5     Iris        10  Decision Tree  0.940000   0.948571  0.940000  0.940219
6     Wine         5    Naive Bayes  0.983016   0.983844  0.983016  0.982961
7     Wine         5            KNN  0.949365   0.956877  0.949365  0.949228
8     Wine         5  Decision Tree  0.876349   0.882125  0.876349  0.876774
9     Wine        10    Naive Bayes  0.977778   0.981411  0.977778  0.977469
10    Wine        10            KNN  0.971895   0.977178  0.971895  0.972372
11    Wine        10  Decision Tree  0.898693   0.906694  0.898693  0.897523
```

In [6]:
```
# Comparison of Holdout vs Cross-Validation
print("\n===== FINAL COMPARISON =====")
print(pd.concat([holdout_df, cv_df], ignore_index=True))
```

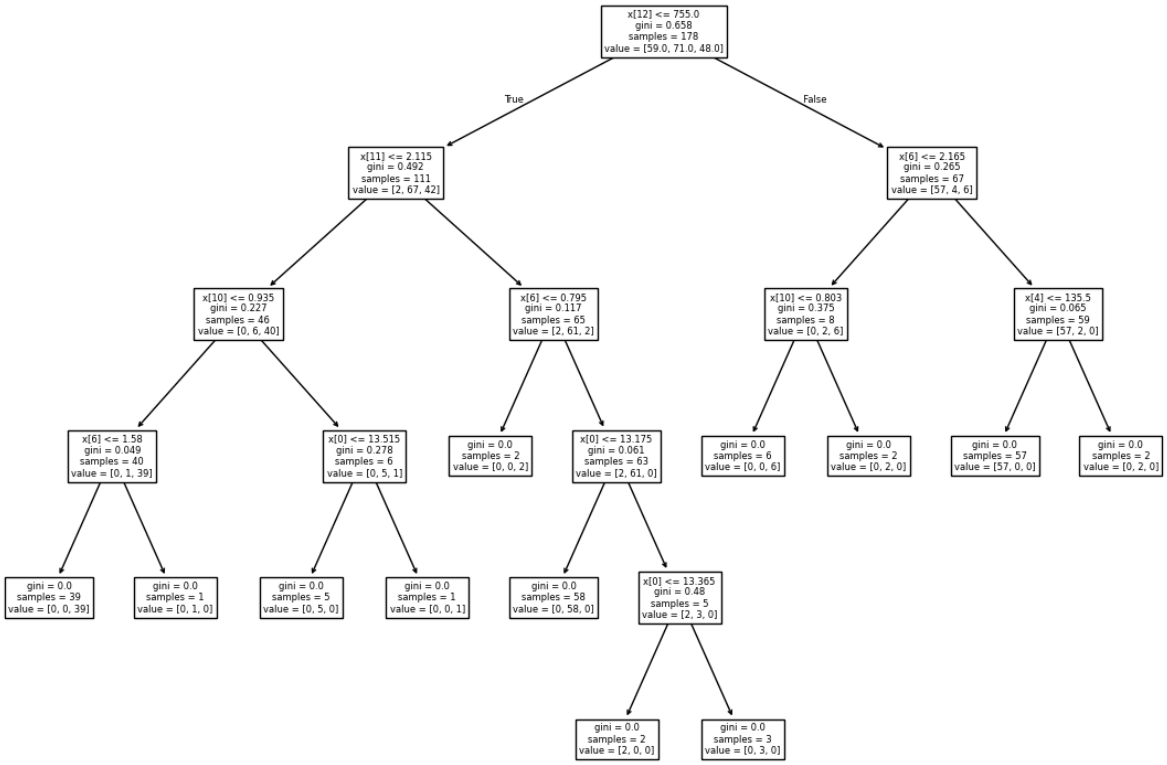
```
===== FINAL COMPARISON =====
   Dataset      Split          Model  Accuracy  Precision    Recall  F1-score  \
0     Iris      80/20    Naive Bayes  0.966667   0.969697  0.966667  0.966583
1     Iris      80/20            KNN  0.933333   0.944444  0.933333  0.932660
2     Iris      80/20  Decision Tree  0.933333   0.933333  0.933333  0.933333
3     Iris  66.6/33.3    Naive Bayes  0.920000   0.923649  0.920000  0.919722
4     Iris  66.6/33.3            KNN  0.920000   0.935238  0.920000  0.918877
5     Iris  66.6/33.3  Decision Tree  0.940000   0.949000  0.940000  0.939529
6     Wine      80/20    Naive Bayes  0.972222   0.974359  0.972222  0.972263
7     Wine      80/20            KNN  0.972222   0.974747  0.972222  0.972369
8     Wine      80/20  Decision Tree  0.944444   0.951389  0.944444  0.944961
9     Wine  66.6/33.3    Naive Bayes  0.966667   0.969231  0.966667  0.966456
10    Wine  66.6/33.3            KNN  0.950000   0.957895  0.950000  0.950476
11    Wine  66.6/33.3  Decision Tree  0.983333   0.984000  0.983333  0.983290
12    Iris         NaN    Naive Bayes  0.960000   0.963879  0.960000  0.959916
13    Iris         NaN            KNN  0.960000   0.962187  0.960000  0.959797
14    Iris         NaN  Decision Tree  0.953333   0.958657  0.953333  0.953185
15    Iris         NaN    Naive Bayes  0.960000   0.967619  0.960000  0.960287
16    Iris         NaN            KNN  0.953333   0.960254  0.953333  0.952893
17    Iris         NaN  Decision Tree  0.940000   0.948571  0.940000  0.940219
18    Wine         NaN    Naive Bayes  0.983016   0.983844  0.983016  0.982961
19    Wine         NaN            KNN  0.949365   0.956877  0.949365  0.949228
20    Wine         NaN  Decision Tree  0.876349   0.882125  0.876349  0.876774
21    Wine         NaN    Naive Bayes  0.977778   0.981411  0.977778  0.977469
22    Wine         NaN            KNN  0.971895   0.977178  0.971895  0.972372
23    Wine         NaN  Decision Tree  0.898693   0.906694  0.898693  0.897523

    CV_Folds
0        NaN
1        NaN
2        NaN
3        NaN
4        NaN
5        NaN
6        NaN
7        NaN
8        NaN
9        NaN
10       NaN
11       NaN
12       5.0
13       5.0
14       5.0
15      10.0
16      10.0
17      10.0
18       5.0
19       5.0
20       5.0
21      10.0
22      10.0
23      10.0
```

In [7]:
```python
# Visualize Decision Tree structures for Iris and Wine
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

```python
# Train a shallow tree for readability (on original features)
dt_iris = DecisionTreeClassifier(random_state=42, max_depth=5)
dt_iris.fit(X1, y1)

plt.figure(figsize=(12, 8))
plot_tree(dt_iris)
plt.title("Decision Tree (Iris)")
plt.show()

dt_wine = DecisionTreeClassifier(random_state=42, max_depth=5)
dt_wine.fit(X2, y2)

plt.figure(figsize=(14, 10))
plot_tree(dt_wine)
plt.title("Decision Tree (Wine)")
plt.show()
```



Decision Tree (Wine)



5. Apply simple K-means algorithm for clustering any dataset. Compare the performance of clusters by varying the algorithm parameters. For a given set of parameters, plot a line graph depicting MSE obtained after each iteration.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
```

In [2]:
```
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Iris Dataset Shape:", X.shape)
print("Features:", iris.feature_names)
print("Target classes:", iris.target_names)
```

```
Iris Dataset Shape: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal widt
h (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']
```

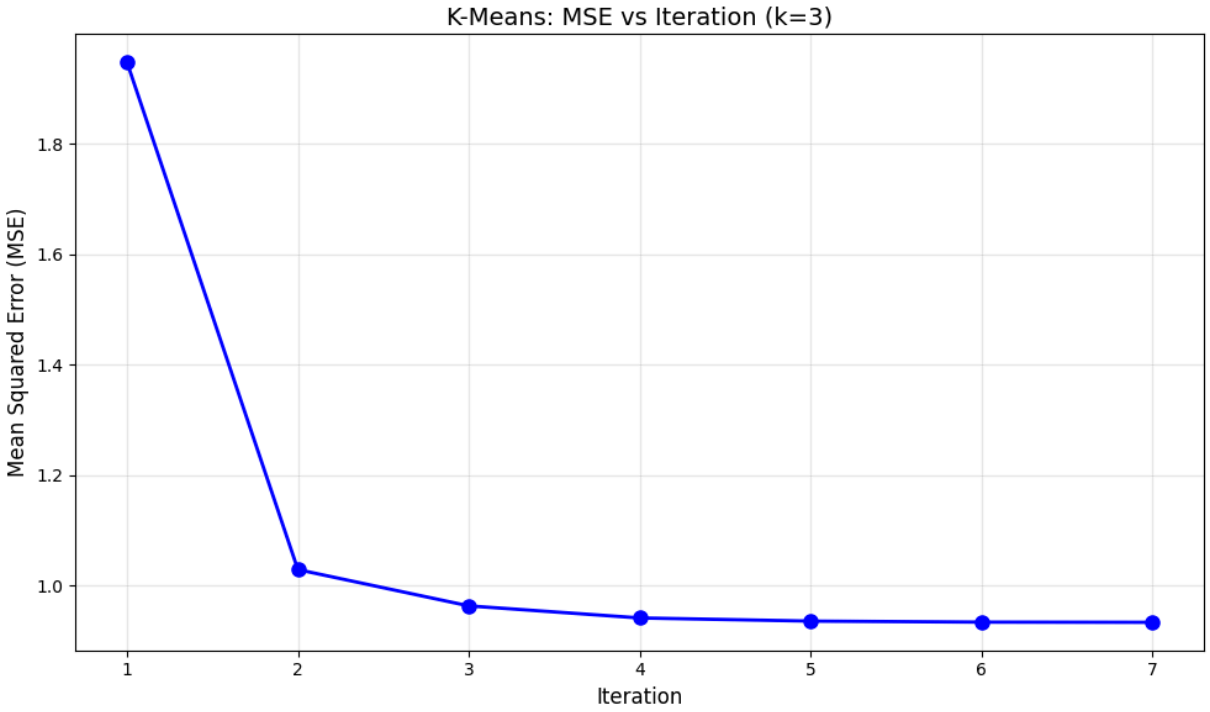
In [3]:
```
def kmeans(X, n_clusters=3, max_iter=100, random_state=42):
    """
    Simple K-Means implementation that tracks MSE per iteration
    Returns: centroids, labels, mse_history
    """
    np.random.seed(random_state)
    n_samples, n_features = X.shape

    # Initialize centroids randomly from the data points
    random_indices = np.random.choice(n_samples, n_clusters, replace=False)
    centroids = X[random_indices].copy()

    mse_history = []
    labels = None

    for iteration in range(max_iter):
        # Assign each point to the nearest centroid
        distances = np.zeros((n_samples, n_clusters))
        for k in range(n_clusters):
            distances[:, k] = np.linalg.norm(X - centroids[k], axis=1)
        labels = np.argmin(distances, axis=1)

        # Calculate MSE for this iteration
        total_error = 0
```

```
        for k in range(n_clusters):
            cluster_points = X[labels == k]
            if len(cluster_points) > 0:
                total_error += np.sum((cluster_points - centroids[k]) ** 2)
        mse = total_error / n_samples
        mse_history.append(mse)

        # Update centroids
        new_centroids = np.zeros((n_clusters, n_features))
        for k in range(n_clusters):
            cluster_points = X[labels == k]
            if len(cluster_points) > 0:
                new_centroids[k] = cluster_points.mean(axis=0)
            else:
                new_centroids[k] = centroids[k]

        # Check for convergence
        if np.allclose(centroids, new_centroids):
            print(f"Converged at iteration {iteration + 1}")
            break

        centroids = new_centroids

    return centroids, labels, mse_history
```

In [4]:
```
# Apply K-Means with k=3 (since Iris has 3 species)
centroids, labels, mse_history = kmeans(X_scaled, n_clusters=3, max_iter=100, rando

print(f"Final MSE: {mse_history[-1]:.4f}")
print(f"Number of iterations: {len(mse_history)}")
```

```
Converged at iteration 7
Final MSE: 0.9336
Number of iterations: 7
```

In [5]:
```
# Plot MSE vs Iteration
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(mse_history) + 1), mse_history, 'b-o', linewidth=2, markersiz
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Mean Squared Error (MSE)', fontsize=12)
plt.title('K-Means: MSE vs Iteration (k=3)', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

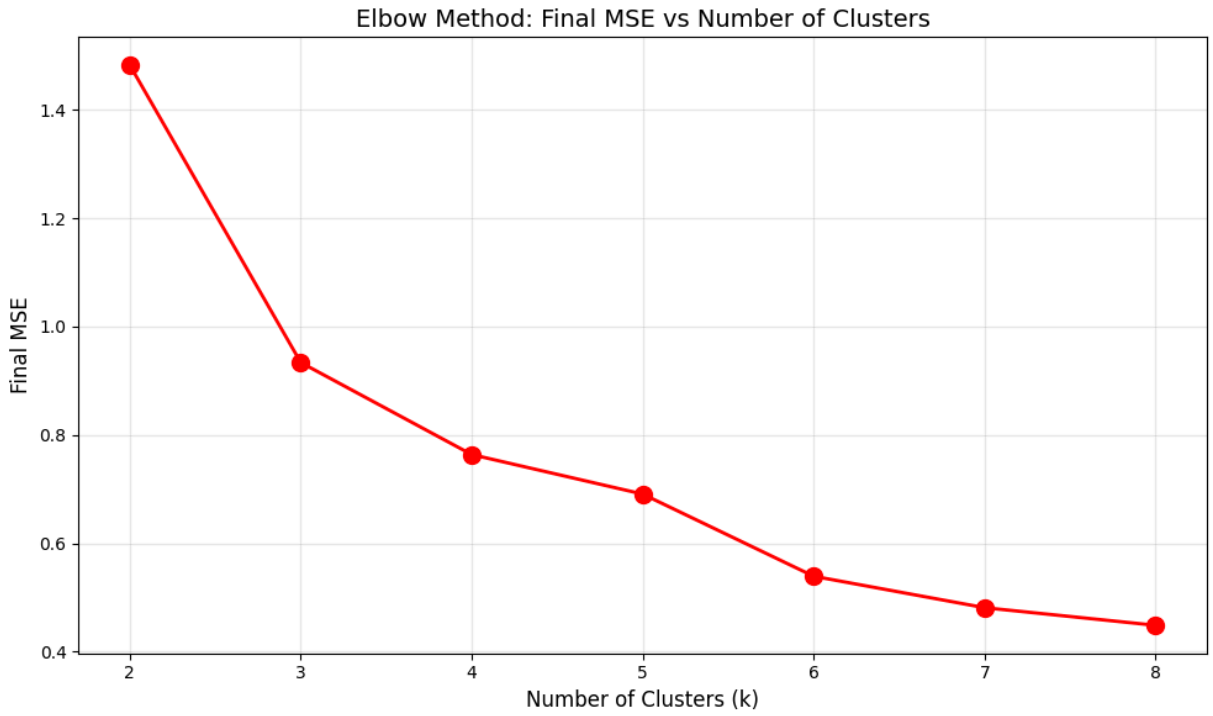


In [6]:
```
k_values = [2, 3, 4, 5, 6, 7, 8]
final_mse_values = []
mse_histories = {}

for k in k_values:
    _, _, hist = kmeans(X_scaled, n_clusters=k, max_iter=100, random_state=42)
    final_mse_values.append(hist[-1])
    mse_histories[k] = hist
    print(f"k={k}: Final MSE = {hist[-1]:.4f}, Iterations = {len(hist)}")
```

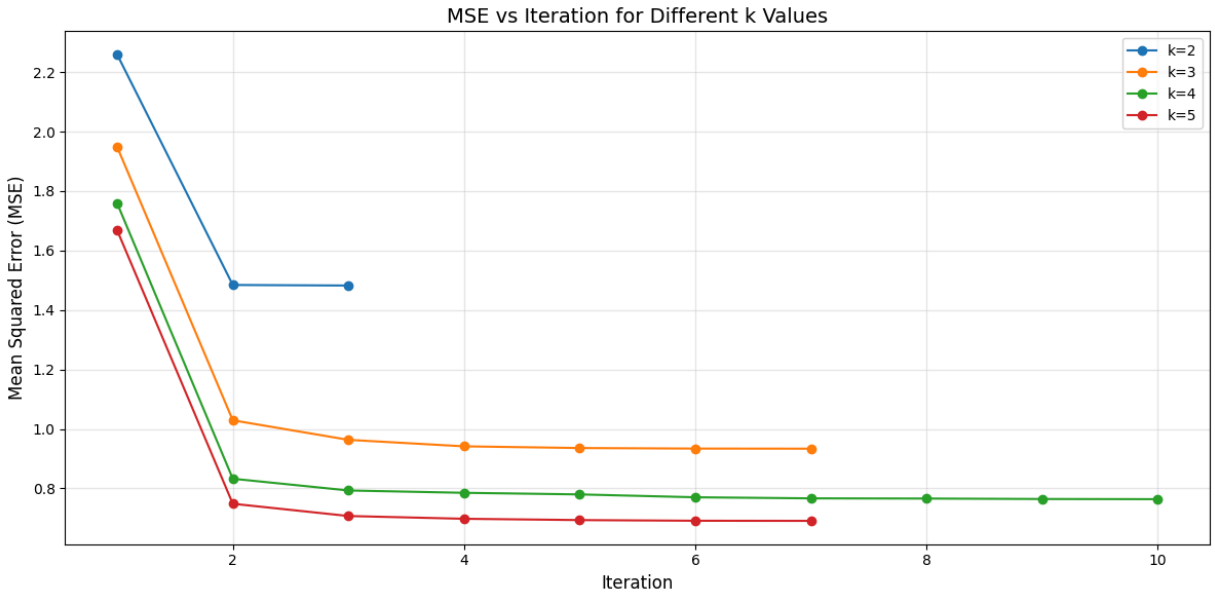
```
Converged at iteration 3
k=2: Final MSE = 1.4824, Iterations = 3
Converged at iteration 7
k=3: Final MSE = 0.9336, Iterations = 7
Converged at iteration 10
k=4: Final MSE = 0.7637, Iterations = 10
Converged at iteration 7
k=5: Final MSE = 0.6908, Iterations = 7
Converged at iteration 9
k=6: Final MSE = 0.5392, Iterations = 9
Converged at iteration 7
k=7: Final MSE = 0.4811, Iterations = 7
Converged at iteration 8
k=8: Final MSE = 0.4486, Iterations = 8
```

In [7]:
```
# Elbow Method: Final MSE vs k
plt.figure(figsize=(10, 6))
plt.plot(k_values, final_mse_values, 'r-o', linewidth=2, markersize=10)
plt.xlabel('Number of Clusters (k)', fontsize=12)
plt.ylabel('Final MSE', fontsize=12)
plt.title('Elbow Method: Final MSE vs Number of Clusters', fontsize=14)
plt.xticks(k_values)
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

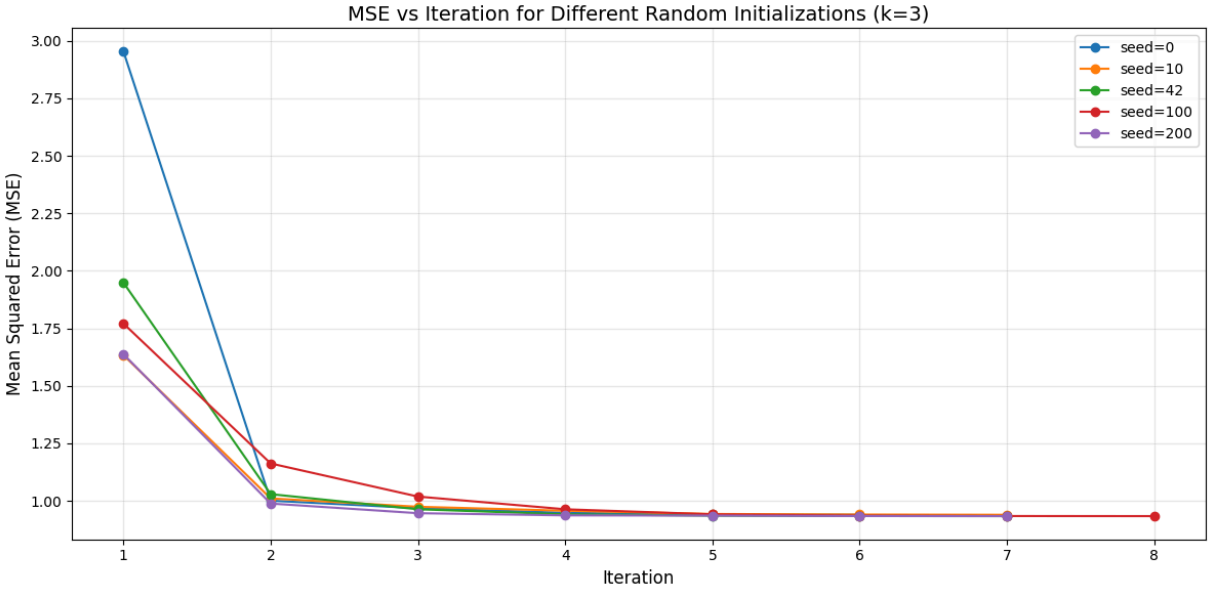


In [8]:
```
# MSE per iteration for different k values
plt.figure(figsize=(12, 6))
for k in [2, 3, 4, 5]:
    plt.plot(range(1, len(mse_histories[k]) + 1), mse_histories[k], '-o', label=f'k
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Mean Squared Error (MSE)', fontsize=12)
plt.title('MSE vs Iteration for Different k Values', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



In [9]:
```
random_states = [0, 10, 42, 100, 200]
init_mse_histories = {}

plt.figure(figsize=(12, 6))
for rs in random_states:
    _, _, hist = kmeans(X_scaled, n_clusters=3, max_iter=100, random_state=rs)
    init_mse_histories[rs] = hist
    print(f"Random State {rs}: Final MSE = {hist[-1]:.4f}, Iterations = {len(hist)}
    plt.plot(range(1, len(hist) + 1), hist, '-o', label=f'seed={rs}', markersize=6)

plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Mean Squared Error (MSE)', fontsize=12)
plt.title('MSE vs Iteration for Different Random Initializations (k=3)', fontsize=1
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

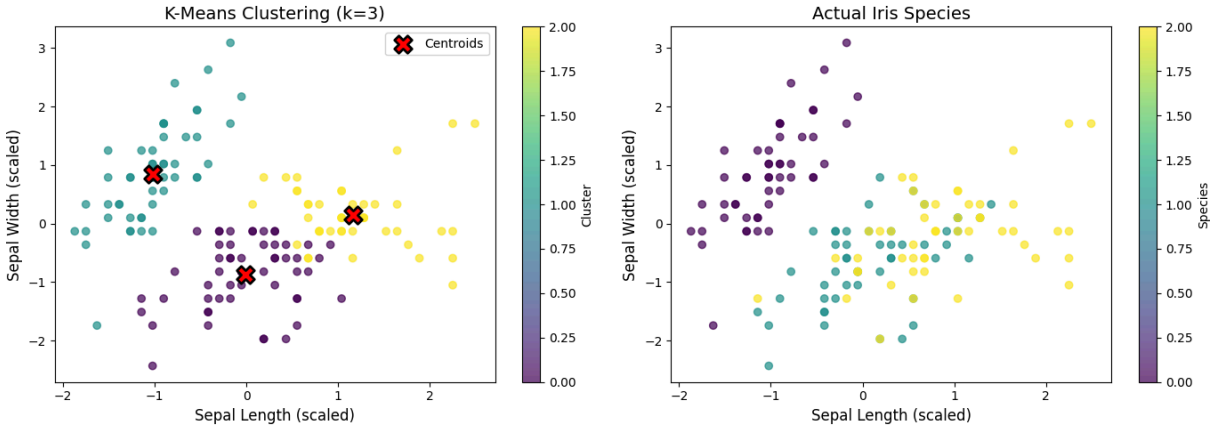
```
Converged at iteration 6
Random State 0: Final MSE = 0.9393, Iterations = 6
Converged at iteration 7
Random State 10: Final MSE = 0.9393, Iterations = 7
Converged at iteration 7
Random State 42: Final MSE = 0.9336, Iterations = 7
Converged at iteration 8
Random State 100: Final MSE = 0.9336, Iterations = 8
Converged at iteration 7
Random State 200: Final MSE = 0.9336, Iterations = 7
```



In [10]:
```
# Final clustering with k=3
final_centroids, final_labels, _ = kmeans(X_scaled, n_clusters=3, max_iter=100, ran

# Plot clusters using first two features
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=final_labels, cmap='viridis
plt.scatter(final_centroids[:, 0], final_centroids[:, 1],
```

```
            c='red', marker='X', s=200, edgecolors='black', linewidths=2, label='Ce
plt.xlabel('Sepal Length (scaled)', fontsize=12)
plt.ylabel('Sepal Width (scaled)', fontsize=12)
plt.title('K-Means Clustering (k=3)', fontsize=14)
plt.legend()
plt.colorbar(scatter, label='Cluster')

plt.subplot(1, 2, 2)
scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='viridis', alpha=0.
plt.xlabel('Sepal Length (scaled)', fontsize=12)
plt.ylabel('Sepal Width (scaled)', fontsize=12)
plt.title('Actual Iris Species', fontsize=14)
plt.colorbar(scatter, label='Species')

plt.tight_layout()
plt.show()
```

```
Converged at iteration 7
```



In [11]:

```
# Summary Table
summary_data = {
    'k': k_values,
    'Final MSE': [f"{mse:.4f}" for mse in final_mse_values],
    'Iterations': [len(mse_histories[k]) for k in k_values]
}
summary_df = pd.DataFrame(summary_data)
print("Performance by Number of Clusters:")
print(summary_df.to_string(index=False))

print("\nConclusions:")
print("- As k increases, MSE decreases (more clusters = less error)")
print("- The 'elbow' in the MSE curve suggests optimal k")
print("- Different initializations can lead to different final solutions")
print("- For Iris dataset, k=3 is optimal (matching 3 species)")
```

```
Performance by Number of Clusters:
 k Final MSE  Iterations
 2    1.4824           3
 3    0.9336           7
 4    0.7637          10
 5    0.6908           7
 6    0.5392           9
 7    0.4811           7
 8    0.4486           8

Conclusions:
- As k increases, MSE decreases (more clusters = less error)
- The 'elbow' in the MSE curve suggests optimal k
- Different initializations can lead to different final solutions
- For Iris dataset, k=3 is optimal (matching 3 species)
```

6. Perform density-based clustering algorithm on a downloaded dataset and evaluate the cluster quality by changing the algorithm's parameters.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bould
from sklearn.neighbors import NearestNeighbors
```

In [2]:
```
# Load and prepare data
df = pd.read_excel('Mall Customers.xlsx')
print("Dataset Shape:", df.shape)
print("Columns:", df.columns.tolist())
df.head()
```

```
Dataset Shape: (200, 7)
Columns: ['CustomerID', 'Gender', 'Age', 'Education ', 'Marital Status', 'Annual Inc
ome (k$)', 'Spending Score (1-100)']
```

Out[2]:

	CustomerID	Gender	Age	Education	Marital Status	Annual Income (k$)	Spending Score (1-100)
0	1	M	19	High School	Married	15	39
1	2	M	21	Graduate	Single	15	81
2	3	F	20	Graduate	Married	16	6
3	4	F	23	High School	Unknown	16	77
4	5	F	31	Uneducated	Married	17	40

In [3]:
```
# Select and scale features
X = df[['Annual Income (k$)', 'Spending Score (1-100)']].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Scaled data shape:", X_scaled.shape)
```

```
Scaled data shape: (200, 2)
```

In [4]:
```
# Helper function to evaluate DBSCAN
def evaluate_dbscan(X, eps, min_samples):
    labels = DBSCAN(eps=eps, min_samples=min_samples).fit_predict(X)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)

    # Calculate silhouette score (needs at least 2 clusters, excluding noise)
    mask = labels != -1
    silhouette = silhouette_score(X[mask], labels[mask]) if n_clusters >= 2 and sum
    return n_clusters, n_noise, silhouette, labels
```

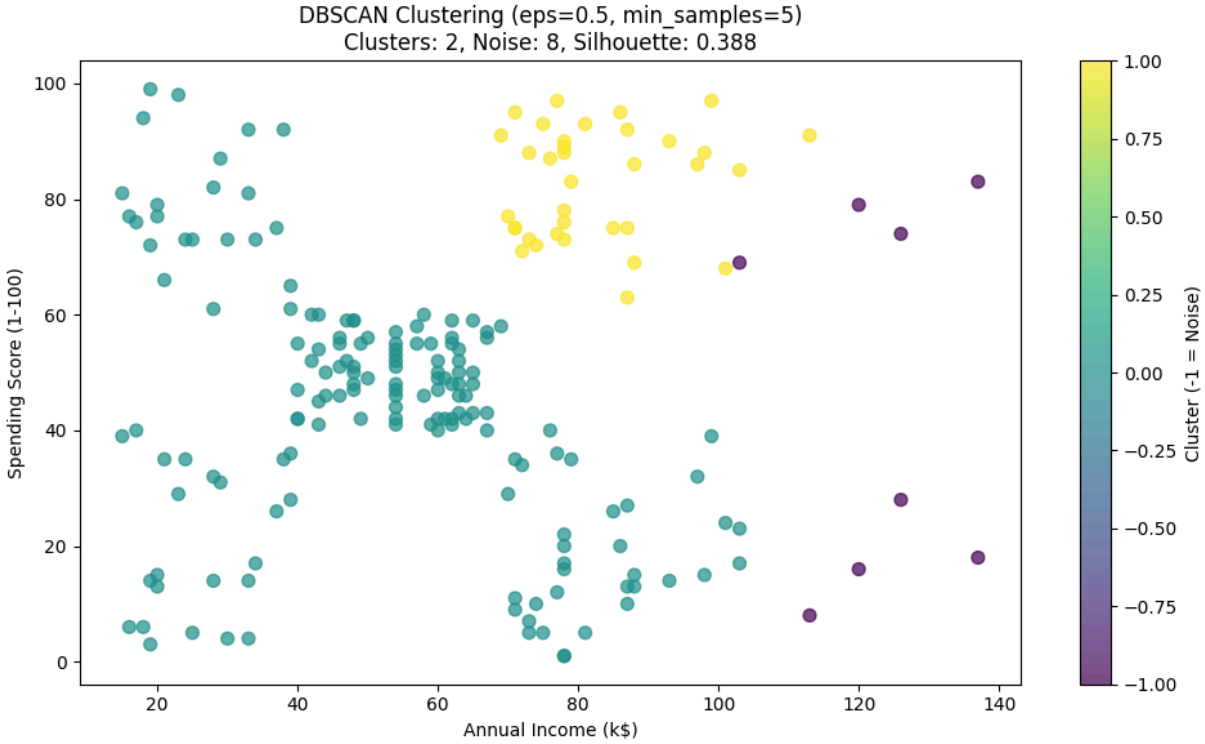
In [5]:
```
# Evaluate by varying eps (epsilon)
print("Effect of varying eps (min_samples=5):")
print("-" * 45)
for eps in [0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    n_clusters, n_noise, sil, _ = evaluate_dbscan(X_scaled, eps, 5)
    sil_str = f"{sil:.3f}" if sil else "N/A"
    print(f"eps={eps}: Clusters={n_clusters}, Noise={n_noise}, Silhouette={sil_str}
```

```
Effect of varying eps (min_samples=5):
---------------------------------------------
eps=0.3: Clusters=7, Noise=35, Silhouette=0.524
eps=0.4: Clusters=4, Noise=15, Silhouette=0.478
eps=0.5: Clusters=2, Noise=8, Silhouette=0.388
eps=0.6: Clusters=1, Noise=5, Silhouette=N/A
eps=0.7: Clusters=1, Noise=0, Silhouette=N/A
eps=0.8: Clusters=1, Noise=0, Silhouette=N/A
```

In [6]:
```
# Evaluate by varying min_samples
print("Effect of varying min_samples (eps=0.5):")
print("-" * 50)
for min_samples in [3, 4, 5, 6, 7, 10]:
    n_clusters, n_noise, sil, _ = evaluate_dbscan(X_scaled, 0.5, min_samples)
    sil_str = f"{sil:.3f}" if sil else "N/A"
    print(f"min_samples={min_samples}: Clusters={n_clusters}, Noise={n_noise}, Silh
```

```
Effect of varying min_samples (eps=0.5):
--------------------------------------------------
min_samples=3: Clusters=2, Noise=7, Silhouette=0.389
min_samples=4: Clusters=2, Noise=8, Silhouette=0.388
min_samples=5: Clusters=2, Noise=8, Silhouette=0.388
min_samples=6: Clusters=2, Noise=11, Silhouette=0.401
min_samples=7: Clusters=2, Noise=12, Silhouette=0.394
min_samples=10: Clusters=4, Noise=21, Silhouette=0.511
```

In [7]:
```
# Visualize final clustering with chosen parameters
eps, min_samples = 0.5, 5
n_clusters, n_noise, sil, labels = evaluate_dbscan(X_scaled, eps, min_samples)

plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', alpha=0.7, s=50)
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={min_samples})\nClusters: {n_
plt.colorbar(label='Cluster (-1 = Noise)')
plt.tight_layout()
plt.show()
```


DBSCAN Clustering (eps=0.5, min_samples=5)
Clusters: 2, Noise: 8, Silhouette: 0.388
Spending Score (1-100)
Annual Income (k$)
Cluster (-1 = Noise)
100
80
60
40
20
0
20
40
60
80
100
120
140
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00